

1)

Os dados obtidos foram à partir dos jogos do país Argentina

As frequências relativas obtidas vitórias do mandante, fH , de empates, fD , e de vitórias do time visitante, fA , são respectivamente

$$fH = 0.441137$$

$$fD = 0.291639$$

$$fA = 0.267224$$

Para determinar a incerteza foi considerado que se pode assumir uma função de probabilidade binomial onde a incerteza é tida como

$$\sqrt{Np(1-p)}$$

Obtendo-se as incertezas:

$$sH = 0.001064$$

$$sD = 0.000945$$

$$sA = 0.000919$$

Logo:

$$fH = 0.4411(10) \text{ frequência de vitórias para Home}$$

$$fD = 0.29163(94) \text{ frequência de empates}$$

$$fA = 0.26722(91) \text{ frequência de vitórias para Away}$$

Supondo que o número de empates em N jogos siga uma binomial com probabilidade individual de ocorrência $\tilde{p} \cong fD$, foi determinada a distribuição esperada de resultados empates em um hipotético “concurso” da Loteria Esportiva com 14 jogos do campeonato considerado.

Com base nesses resultados, a chance de haver menos de dois empates em um “concurso” dessa Loteria é 0.054170860616354906

O resultado é obtido realizando uma distribuição de probabilidade binomial e somando os resultados

Assim, também, a probabilidade de haver empate em mais da metade dos jogos de um “concurso” é 0.08172361316898669

2)

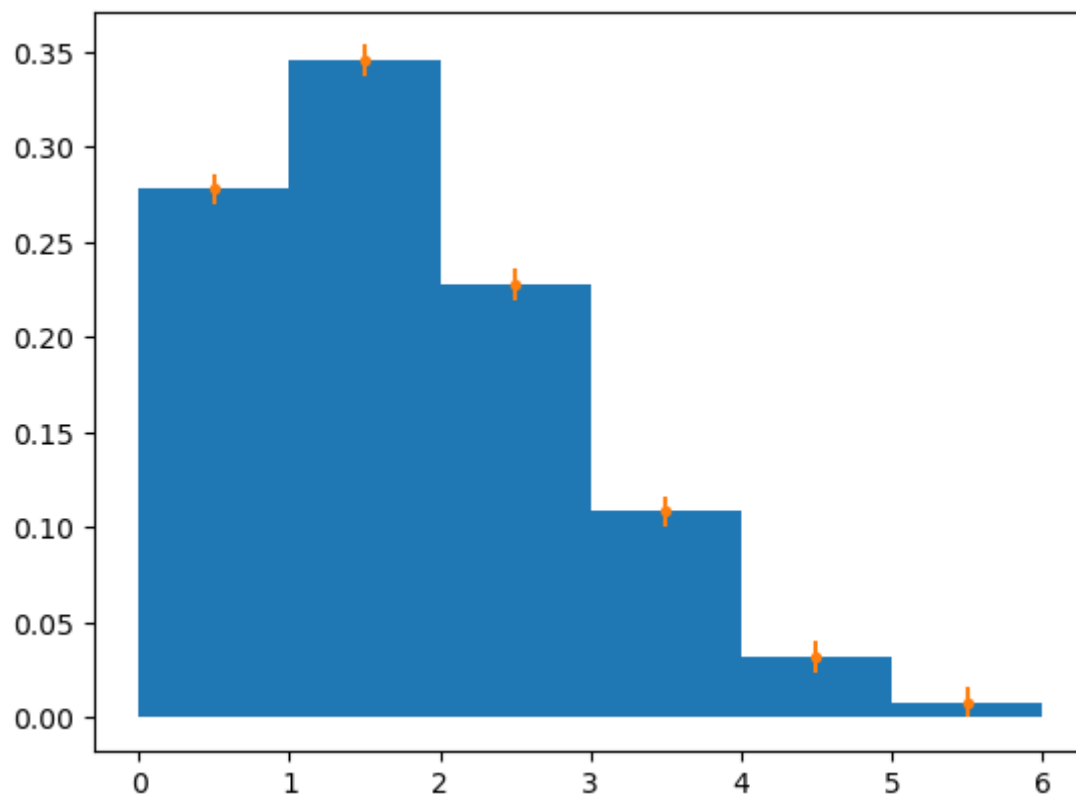
Média do Número de gols

	HG	AG
média	1.295987	0.980268
std	1.129467	1.031105
incerteza	0.021	0.019

$$\text{Gols de HG} = 1.296(0.021)$$

$$\text{Gols de AG} = 1.296(0.021)$$

Figura 1

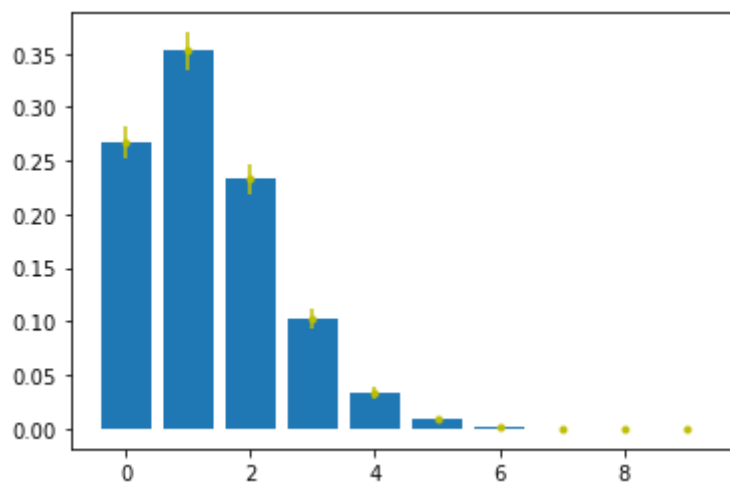


Histograma da frequência relativa do número de gols de Home

A figura 1 corresponde ao histograma dos gols de Home, suas incertezas foram obtidas considerando o histograma uma binomial com incerteza para cada *bin* estimada por

$$\sqrt{Np(1-p)}$$

Figura 2

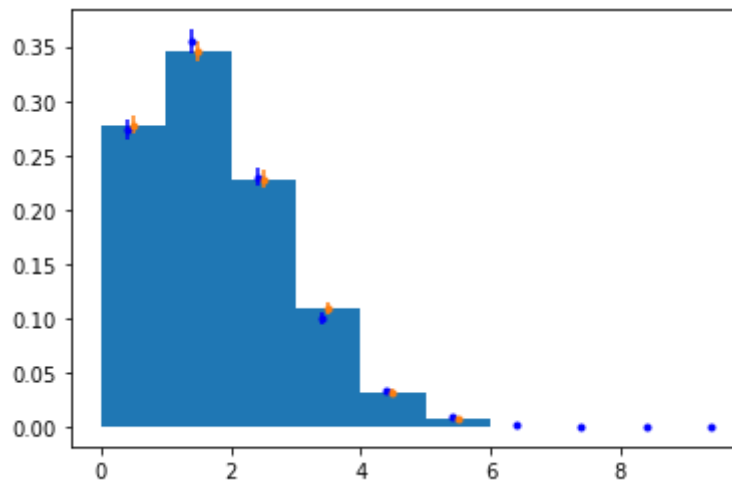


Distribuição de Poisson pelo número médio de gols de Home

As incertezas da figura 2 estimadas foram obtidas pela raiz quadrada do numero de contagens para cada *bin*, a poisson foi programada a partir da media de gols de Home

Ao se comparar o histograma da frequência relativa com os pontos e incertezas estimados para uma distribuição de poisson obtemos a figura 3

Figura 3



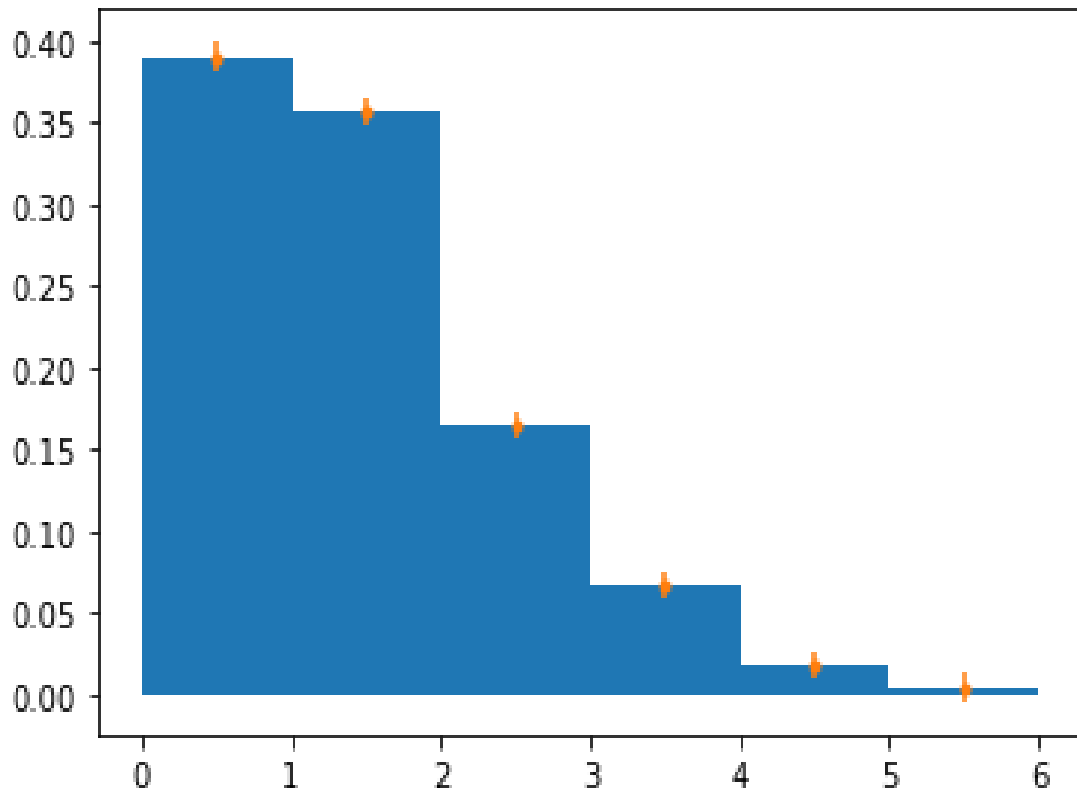
Comparação histograma e Poisson estipulada(pontos azuis)

Testes Z

*[-0.17911938, 0.38825078, 0.06968549, -0.36151006, 0.00216589,
0.01199876]*

Na figura 3 todos os pontos de poisson e do histograma, tomando suas respectivas incertezas, calculados encontram-se à $|\Delta| \leq 3\sigma\Delta$ que estão explícitos no gráfico, logo passam num teste Z com nível de significância de $\sim 0,3\%$, assim podemos assumir que uma distribuição de Poisson a partir da média de gols para Home é uma estipulação adequada

Figura 4

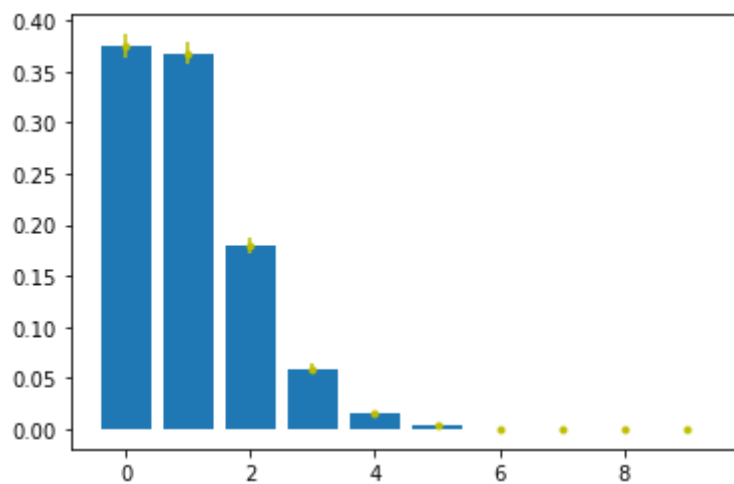


Histograma da frequência relativa do número de gols de Away

A figura 4 corresponde ao histograma dos gols de Away, suas incertezas foram obtidas considerando o histograma uma binomial com incerteza para cada *bin* estimada por

$$\sqrt{Np(1-p)}$$

Figura 5

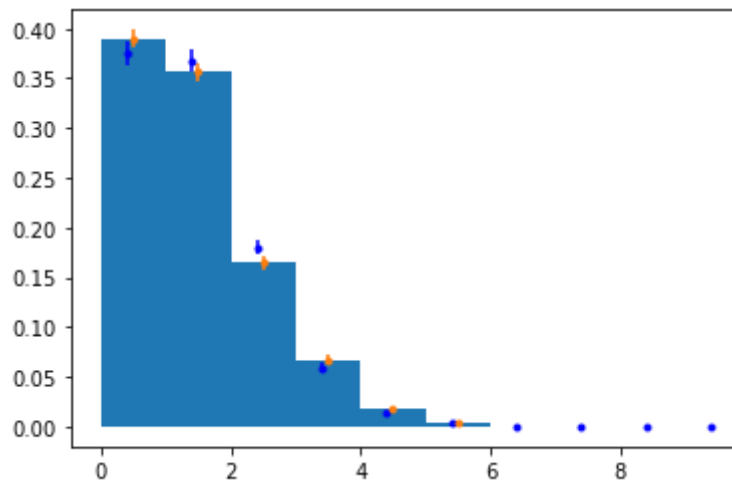


Distribuição de Poisson pelo número médio de gols de Away

As incertezas da figura 5 estimadas foram obtidas pela raiz quadrada do numero de contagens para cada *bin*, a poisson foi programada a partir da media de gols de Away

Ao se comparar o histograma da freqüência relativa com os pontos e incertezas estimados para uma distribuição de poisson obtemos a figura 6

Figura 6



Comparação histograma e Poisson estipulada(pontos azuis)

Testes Z

[-0.63589102, 0.48134922, 0.63472344, -0.31399318, -0.12777215, -0.05874325])

Na figura 6 todos os pontos de poisson e do histograma, tomando suas respectivas incertezas, calculados encontram-se à $|\Delta| \leq 3\sigma\Delta$ que estão explícitos no gráfico, logo passam num teste Z com nível de significância de $\sim 0,3\%$, assim podemos assumir que uma distribuição de Poisson a partir da média de gols para Away é uma estipulação adequada.

Considerando agora o número total de gols, $T = HG + AG$, e o saldo de gols, $S = HG - AG$, de cada partida.

Para S, obtemos :

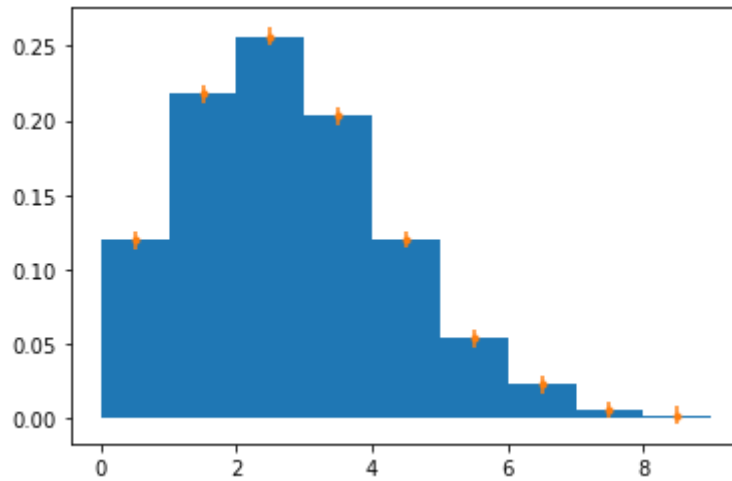
N 2990
 Sm 0.315719
 σ_s 1.523098
 σ_m 0.027854
 Logo, numero médio de gols de S é 0.316(28) gols

Para T, obtemos :

N 2990
 Tm 2.276254
 σ_t 1.535551
 σ_m 0.028082
 Logo, numero médio de gols de T é 2.276(28) gols

Fazendo um histograma (figura 7), em frequência relativa, do número total de gols marcados por partida (para $n=0, 1, \dots, 9$ gols) com suas respectivas incertezas e comparando com a previsão de uma distribuição de Poisson com parâmetro T_m .

Figura 7



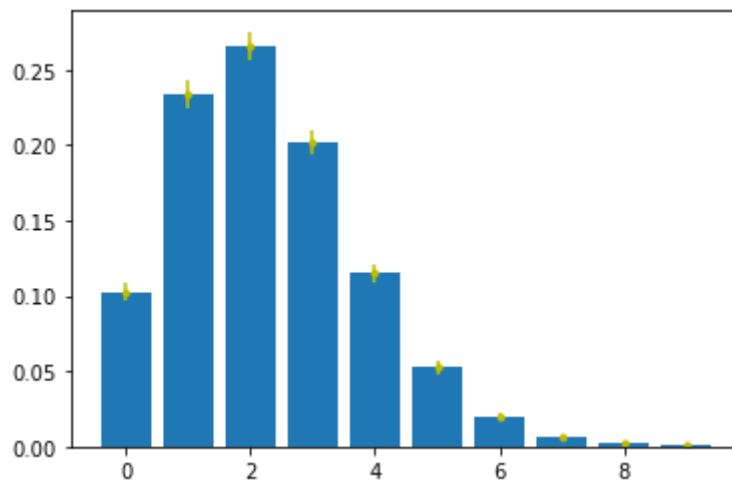
Histograma em frequência relativa do número total de gols de T

Para figura 6, suas incertezas foram obtidas considerando o histograma uma binomial com incerteza para cada *bin* estimada por

$$\sqrt{Np(1-p)}$$

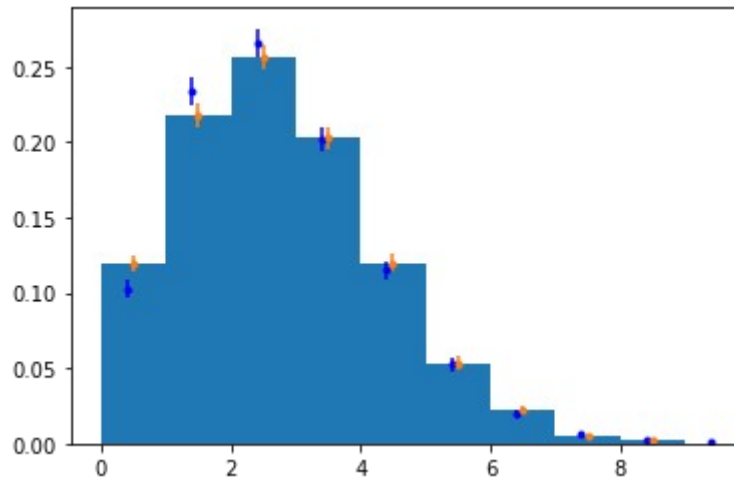
Ao se comparar o histograma da frequência relativa com os pontos e incertezas estimados para uma distribuição de poisson obtemos a figura 8

Figura 8



Distribuição de Poisson pelo número médio de gols de T

Figura 9



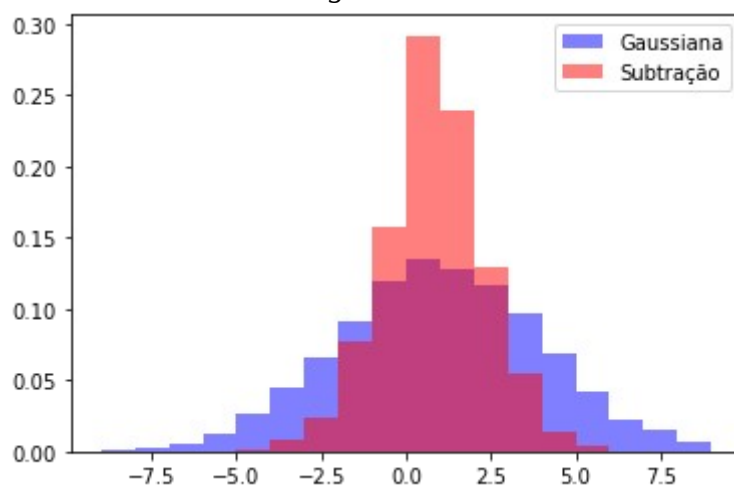
Comparação histograma e Poisson estipulada (Pontos azuis)

Testes Z

$[-0.66708514, 0.65423067, 0.39163339, -0.04888013, -0.20830547, -0.04816903, -0.10005723, 0.04253793, -0.0066344]$

Na figura 9 todos os pontos de Poisson e do histograma, tomando suas respectivas incertezas, calculados encontram-se à $|\Delta| \leq 3 \Delta$ que estão explícitos no gráfico, logo passam num teste Z com nível de significância de $\sim 0,3\%$, assim podemos assumir que uma distribuição de Poisson a partir da média de gols para Away é uma estipulação adequada.

Figura 10



Comparação de histograma de S e de uma gaussiana

Na figura 10 foi feita uma sobreposição do histograma de S e de uma gaussiana de mesmos valores de desvio padrão e valor médio de S.

Para compara-los, além da diferença visual da sobreposição dos gráficos, foi obtido a curtose de S e sua assimetria.

Assimetria de S = -0.06474345442608358

Curtose de S = 0.3518631483774901

Visualmente S difere de uma gaussiana, sua curtose 0.35 é muito menor que a curtose 3 de uma gaussiana, o que implica uma queda mais abrupta conforme mais distante da média do que a de uma gaussiana. A sua assimetria próxima de 0 se assemelha a simetria de uma gaussiana.

Para S, os valores entre $S_{m+1\sigma}$, $S_{m+2\sigma}$, $S_{m+3\sigma}$ são

$S_{m+1\sigma}$ 0.69%

$S_{m+2\sigma}$ 0.97%

$S_{m+3\sigma}$ 0.99%

Ainda assim, S não pode ser definida apropriadamente como uma gaussiana.

Considerando que se o número de gols marcados pelo mandante e pelo visitante forem estatisticamente independentes os desvios-padrões amostrais de T e de S (respectivamente, s_T e s_S) deveriam ser equivalentes.

Tomando o a incerteza do desvio padrão amostral como :

$$\sigma_{s\#} \cong \frac{s\#}{\sqrt{2(N-1)}}.$$

	σ_s	$\sigma_{s\#}$
σ_s	1.523098	0.019699
σ_t	1.535551	0.019860

Logo $\sigma_s=1.523(20)$ gol e $\sigma_t=1.536(20)$ gol

Usando teste Z, temos -24.34, recusando no teste de compatibilidade.

Estimando a covariância entre S e T por

$$(\sigma_{x+y})^2 - (\sigma_{x-y})^2 = 4cov(x, y)$$

Logo:

$$cov(HG, AG) = \frac{(\sigma_T)^2 - (\sigma_S)^2}{4}$$

$cov(HG, AG) = 0.009522 \text{ gol}^2$

tomando

$$\rho_{HG, AG} = \frac{cov(HG, AG)}{s_{HG} s_{AG}}.$$

Obtemos $p=0.004071$

Um coeficiente de correlação tão pequeno implica pouca dependência entre S e T, entretanto para afirmar total independência p deveria ser 0, ainda assim está muito próximo de 0.

Utilizando as funções nativas do python para covariância e coeficiente de correlação obtemos: $cov(HG, AG) = 0.21251792 \text{ gol}^2$ e $p=0.09086645$.

Estes valores diferem das atribuições matemáticas assumidas anteriormente, aparentemente há fatores que atribuem correlação entre S e T que não podem ser bem determinados somente pelas análises matemáticas envolvidas na covariância da soma e da subtração.

Os dados utilizados até aqui não envolviam os jogos pós pandemia, daqui em diante será avaliado estes jogos e se estatisticamente são discrepantes dos resultados de até agora.

Caso estatisticamente não sejam inferidos diferenças em ambos os períodos,

Frquências obtidas

H 0.414654

D 0.294754

A 0.290591

Incertezas

H 0.002174

D 0.001969

A 0.001960

Logo

fH = 0.4147(21) frequencia de vitórias para Home

fD = 0.2948(20) frequencia de empates

fA = 0.2906(20) frequencia de vitórias para Away

Testes Z

fH -10.94

fD 1.42

fA 10.79

Saldo de gol HG 4.105395630193197

Somente a frequencia de empates antes e depois da pandemia podem ser tidos estatisticamente como correspondentes.

In [1]:

```
import pandas as pd
import scipy.stats as st
import matplotlib.pyplot as plt
import numpy as np
```

In [2]:

```

tabela = pd.read_csv('./argentina.csv', sep=';', header=0) #transforma sua planilha em uma tabela
print(tabela, '\n')

#é muito mais fácil trabalhar com
#matrizes, então convertamos para uma
#matriz do tipo numpy nessas linhas

data = tabela.to_numpy()
print(data, '\n')

gol_h = data[:,7] #aqui estamos pegando as colunas com os dados relevantes
gol_a = data[:,8] #os ':' pedem para pegar todas as linhas e o número depois da
res = data[:,9]   #vírgula indica o número da coluna pra pegar os dados

print(gol_h)
print(gol_a)
print(res)

```

	Country	League	Season	Date	Time	\
0	Argentina	Liga Profesional	2012/2013	8/3/2012	23:00	
1	Argentina	Liga Profesional	2012/2013	8/4/2012	1:10	
2	Argentina	Liga Profesional	2012/2013	8/4/2012	18:10	
3	Argentina	Liga Profesional	2012/2013	8/4/2012	20:10	
4	Argentina	Liga Profesional	2012/2013	8/4/2012	22:15	
...	...	...	...	...	...	
4288	Argentina	Liga Profesional	2022	10/18/2022	23:00	
4289	Argentina	Liga Profesional	2022	10/19/2022	1:30	
4290	Argentina	Liga Profesional	2022	10/19/2022	1:30	
4291	Argentina	Liga Profesional	2022	10/20/2022	1:30	
4292	Argentina	Liga Profesional	2022	10/20/2022	19:00	

	Home	Away	HG	AG	Res	PH	PD	PA	Max
H \									
0	Arsenal Sarandi	Union de Santa Fe	1	0	H	1.90	3.39	5.03	1.9
0									
1	Velez Sarsfield	Argentinos Jrs	3	0	H	2.00	3.24	4.55	2.1
8									
2	Racing Club	Atl. Rafaela	1	1	D	1.91	3.34	5.09	2.0
0									
3	Colon Santa FE	Lanus	1	0	H	2.50	3.16	3.27	2.5
4									
4	Quilmes	Boca Juniors	3	0	H	3.18	3.13	2.58	3.3
7									
...	...	...	..	..	..	...	...	...	
...									
4288	Lanus	Racing Club	0	1	A	5.55	3.76	1.69	5.9
5									
4289	Huracan	Platense	2	0	H	1.65	3.58	6.66	1.6
5									
4290	Central Cordoba	Tigre	1	2	A	3.23	3.26	2.40	3.2
3									
4291	Talleres Cordoba	Union de Santa Fe	2	2	D	1.78	3.54	5.21	2.0
0									
4292	Gimnasia L.P.	Boca Juniors	1	2	A	3.13	3.03	2.61	3.7
0									
	MaxD	MaxA	AvgH	AvgD	AvgA				
0	3.50	5.68	1.76	3.30	4.74				
1	3.30	4.62	1.97	3.12	3.96				
2	3.40	5.09	1.91	3.22	4.05				
3	3.16	3.36	2.39	2.99	3.04				

```
4      3.13  2.58  3.07  2.99  2.38
...      ...      ...      ...      ...      ...
4288  3.81  1.75  5.17  3.60  1.69
4289  3.65  6.75  1.63  3.50  6.19
4290  3.30  2.50  3.06  3.16  2.38
4291  3.60  5.21  1.81  3.44  4.59
4292  3.00  2.55  3.29  2.90  2.45
```

[4293 rows x 19 columns]

```
[['Argentina' 'Liga Profesional ' '2012/2013' ... 1.76 3.3 4.74]
['Argentina' 'Liga Profesional ' '2012/2013' ... 1.97 3.12 3.96]
['Argentina' 'Liga Profesional ' '2012/2013' ... 1.91 3.22 4.05]
...
['Argentina' 'Liga Profesional' '2022' ... 3.06 3.16 2.38]
['Argentina' 'Liga Profesional' '2022' ... 1.81 3.44 4.59]
['Argentina' 'Liga Profesional' '2022' ... 3.29 2.9 2.45]]

[1 3 1 ... 1 2 1]
[0 0 1 ... 2 2 2]
['H' 'H' 'D' ... 'A' 'D' 'A']
```

In [3]:

tabela

Out[3]:

	Country	League	Season	Date	Time	Home	Away	HG	AG	Res
0	Argentina	Liga Profesional	2012/2013	8/3/2012	23:00	Arsenal Sarandi	Union de Santa Fe	1	0	H
1	Argentina	Liga Profesional	2012/2013	8/4/2012	1:10	Velez Sarsfield	Argentinos Jrs	3	0	H
2	Argentina	Liga Profesional	2012/2013	8/4/2012	18:10	Racing Club	Atl. Rafaela	1	1	D
3	Argentina	Liga Profesional	2012/2013	8/4/2012	20:10	Colon Santa FE	Lanus	1	0	H
4	Argentina	Liga Profesional	2012/2013	8/4/2012	22:15	Quilmes	Boca Juniors	3	0	H
...	...	...	...	...	...	...	...	...	...	...
4288	Argentina	Liga Profesional	2022	10/18/2022	23:00	Lanus	Racing Club	0	1	A
4289	Argentina	Liga Profesional	2022	10/19/2022	1:30	Huracan	Platense	2	0	H
4290	Argentina	Liga Profesional	2022	10/19/2022	1:30	Central Cordoba	Tigre	1	2	A
4291	Argentina	Liga Profesional	2022	10/20/2022	1:30	Talleres Cordoba	Union de Santa Fe	2	2	D
4292	Argentina	Liga Profesional	2022	10/20/2022	19:00	Gimnasia L.P.	Boca Juniors	1	2	A

4293 rows x 19 columns



In [4]:

```
tabela[:][:2990]
```

Out[4]:

	Country	League	Season	Date	Time	Home	Away	HG	AG	Res
0	Argentina	Liga Profesional	2012/2013	8/3/2012	23:00	Arsenal Sarandi	Union de Santa Fe	1	0	H
1	Argentina	Liga Profesional	2012/2013	8/4/2012	1:10	Velez Sarsfield	Argentinos Jrs	3	0	H
2	Argentina	Liga Profesional	2012/2013	8/4/2012	18:10	Racing Club	Atl. Rafaela	1	1	D
3	Argentina	Liga Profesional	2012/2013	8/4/2012	20:10	Colon Santa FE	Lanus	1	0	H
4	Argentina	Liga Profesional	2012/2013	8/4/2012	22:15	Quilmes	Boca Juniors	3	0	H
...	...	...	...	...	...	...	...	...	...	...
2985	Argentina	Liga Profesional	2019/2020	12/1/2019	0:45	Newells Old Boys	River Plate	2	3	A
2986	Argentina	Liga Profesional	2019/2020	12/1/2019	20:10	San Lorenzo	Patronato	2	0	H
2987	Argentina	Liga Profesional	2019/2020	12/1/2019	22:40	Godoy Cruz	Talleres Cordoba	0	5	A
2988	Argentina	Liga Profesional	2019/2020	12/1/2019	22:40	Velez Sarsfield	Colon Santa FE	3	1	H
2989	Argentina	Liga Profesional	2019/2020	12/2/2019	0:45	Aldosivi	Independiente	0	0	D

2990 rows × 19 columns

In [5]:

```
df=tabela[:][:2990]
```

In [6]:

```
freq=df['Res'].value_counts()/sum(df['Res'].value_counts())
```

In [7]:

```
freq
```

Out[7]:

H 0.441137
D 0.291639
A 0.267224
Name: Res, dtype: float64

In [8]:

```
df['Res'].value_counts()
```

Out[8]:

```
H    1319
D     872
A     799
Name: Res, dtype: int64
```

In [9]:

```
'incertezas'
((df['Res'].value_counts()*freq*(1-freq))**.2)/2990
```

Out[9]:

```
H    0.001064
D    0.000945
A    0.000919
Name: Res, dtype: float64
```

In [10]:

```
df.describe()
```

Out[10]:

	HG	AG	PH	PD	PA	MaxH	M
count	2990.000000	2990.000000	2987.000000	2987.000000	2987.000000	2990.000000	2990.000000
mean	1.295987	0.980268	2.454921	3.365775	4.208614	2.527385	3.429834
std	1.129467	1.031105	0.856784	0.482814	1.980018	0.899664	0.497389
min	0.000000	0.000000	1.150000	2.050000	1.410000	1.170000	2.350000
25%	0.000000	0.000000	1.900000	3.100000	2.930000	1.950000	3.150000
50%	1.000000	1.000000	2.260000	3.240000	3.750000	2.320000	3.300000
75%	2.000000	2.000000	2.770000	3.475000	4.905000	2.850000	3.527500
max	6.000000	6.000000	8.870000	9.020000	23.920000	9.500000	9.020000

In [11]:

```
b=[]
for i in range(1,15):
    b.append(st.binom(i,freq[1]).pmf(0))
print(b)
```

```
[0.708361204013378, 0.5017755953512825, 0.355438364867564, 0.25177874809013
4, 0.17835029714210832, 0.12633643121972757, 0.08949182652955955, 0.06339253
798983516, 0.04490481453594344, 0.03180882849067833, 0.022532140047911944,
0.015960893853336957, 0.01130607798707949, 0.008008787015596777]
```

In [12]:

```
b=[]  
for i in range(1,15):  
    b.append(st.binom.pmf(0,i,freq[1]))  
print(b)
```

```
[0.708361204013378, 0.5017755953512825, 0.355438364867564, 0.25177874809013  
4, 0.17835029714210832, 0.12633643121972757, 0.08949182652955955, 0.06339253  
798983516, 0.04490481453594344, 0.03180882849067833, 0.022532140047911944,  
0.015960893853336957, 0.01130607798707949, 0.008008787015596777]
```

In [13]:

```
sum(b)
```

Out[13]:

```
2.409446547134136
```

In [14]:

```
#prob em menos de 2 jogos  
b=[]  
for i in range(2):  
    b.append(st.binom.pmf(i,14,freq[1]))  
print(sum(b))
```

```
0.05417086061635478
```

In [15]:

```
#prob em mais de 7 jogos  
b=[]  
for i in range(7,15):  
    b.append(st.binom.pmf(i,14,freq[1]))  
print(sum(b))
```

```
0.08172361316898652
```

In [16]:

```
mean= df[['HG', 'AG']].describe()[1:2]
df[['HG', 'AG']].describe()[1:2]
```

Out[16]:

	HG	AG
count	2990.000000	2990.000000
mean	1.295987	0.980268
std	1.129467	1.031105
min	0.000000	0.000000
25%	0.000000	0.000000
50%	1.000000	1.000000
75%	2.000000	2.000000
max	6.000000	6.000000

In [17]:

```
df[['HG', 'AG']].describe().iloc[2][1]
```

Out[17]:

1.0311047040222532

In [18]:

```
df[['HG', 'AG']].describe().iloc[1][0]**.5
```

Out[18]:

1.1384140819901951

In [19]:

```
df[['HG', 'AG']].describe().iloc[1][0]/(2*2989)**.5
```

Out[19]:

0.016761873670079277

In [20]:

```
df[['HG', 'AG']].describe().iloc[1][0]**.5-df[['HG', 'AG']].describe().iloc[1][0]/(2*2989)**.5
```

Out[20]:

1.121652208320116

In [21]:

```
df[['HG', 'AG']].describe().iloc[1][1]**.5
```

Out[21]:

0.9900846219028089

In [22]:

```
df[['HG', 'AG']].describe().iloc[1][1]/(2*2989)**.5
```

Out[22]:

0.012678464961807061

In [23]:

```
df[['HG', 'AG']].describe().iloc[1][1]**.5-df[['HG', 'AG']].describe().iloc[1][1]/(2*2989)**.5
```

Out[23]:

0.9774061569410019

In [24]:

```
'desv padrao da media'
df[['HG', 'AG']].describe().iloc[2][0]/(2989)**.5
```

Out[24]:

0.02065905442616808

In [25]:

```
df[['HG', 'AG']].describe().iloc[2][1]/(2989)**.5
```

Out[25]:

0.018859918455305925

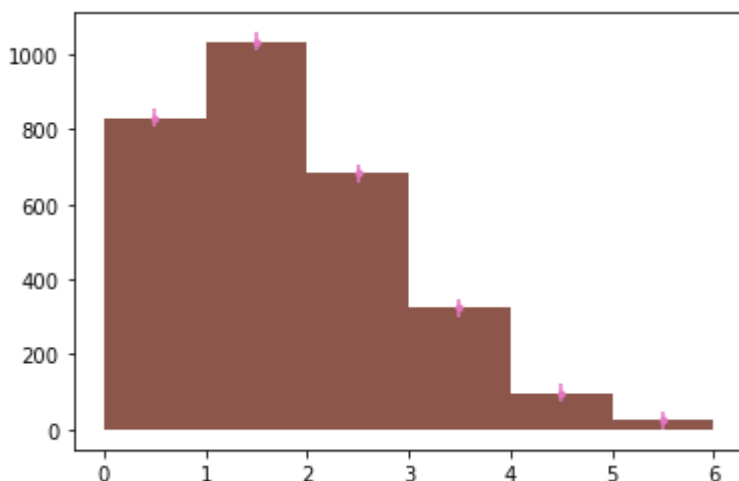
In [69]:

```
ax=plt.hist(df['HG'],6)
erro=((plt.hist(df['HG'],6)[0]*(1-plt.hist(df['HG'],6)[0]/sum(plt.hist(df['HG'],6)[0])))**.
plt.errorbar(plt.hist(df['HG'],6)[1][: -1]+.5,plt.hist(df['HG'],6)[0],yerr=erro[0],fmt='.')

```

Out[69]:

<ErrorbarContainer object of 3 artists>



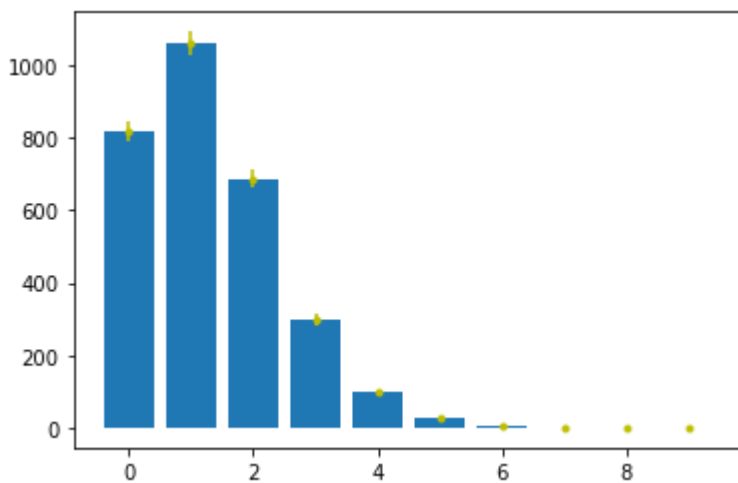
In [70]:

```
b=[]
for i in range(10):
    b.append(st.poisson.pmf(i,mean['HG'][0])*2990)
plt.bar(range(10),b)

plt.errorbar(np.arange(0,10,1),b,yerr=[i**.5 for i in b],fmt='.y')
b=[]
for i in range(10):
    b.append(st.poisson.pmf(i,mean['HG'][0])*2990)
[i**.5 for i in b]
```

Out[70]:

```
[28.6032692906749,
 32.562364551462004,
 26.212062544555838,
 17.228236601566017,
 9.806433577540824,
 4.9925951228265,
 2.3203365559079026,
 0.9983945955749348,
 0.4018439991051225,
 0.15248828911484233]
```

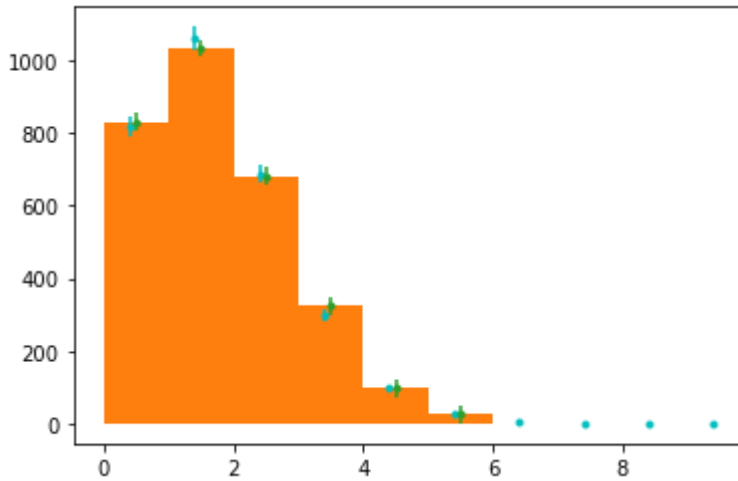


In [71]:

```
plt.errorbar(np.arange(.4,10,1),b,yerr=[i**.5 for i in b],fmt='.c')
plt.errorbar(plt.hist(df['HG'],6)[1][: -1]+.5,plt.hist(df['HG'],6)[0],yerr=erro[0],fmt='.')
```

Out[71]:

<ErrorbarContainer object of 3 artists>

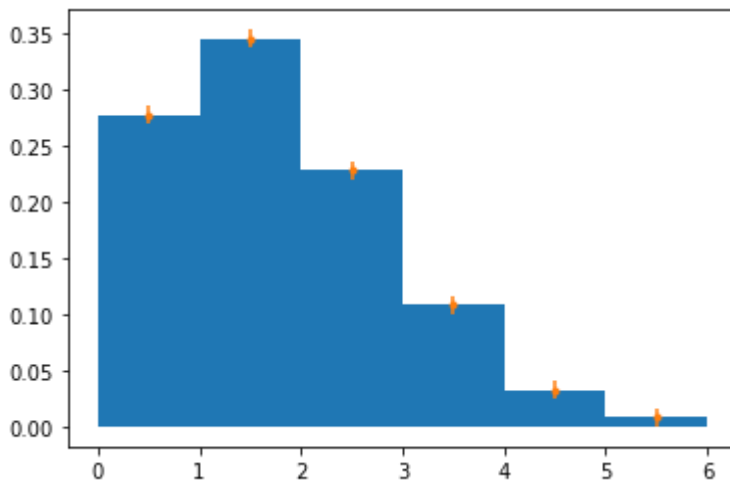


In [72]:

```
ax=plt.hist(df['HG'],6,density=True)
erro=((ax[0]*(1-ax[0]/sum(ax[0])))**.5)
plt.errorbar(ax[1][: -1]+.5,ax[0],yerr=erro[0]/2990**.5,fmt='.')
```

Out[72]:

<ErrorbarContainer object of 3 artists>



In [73]:

```

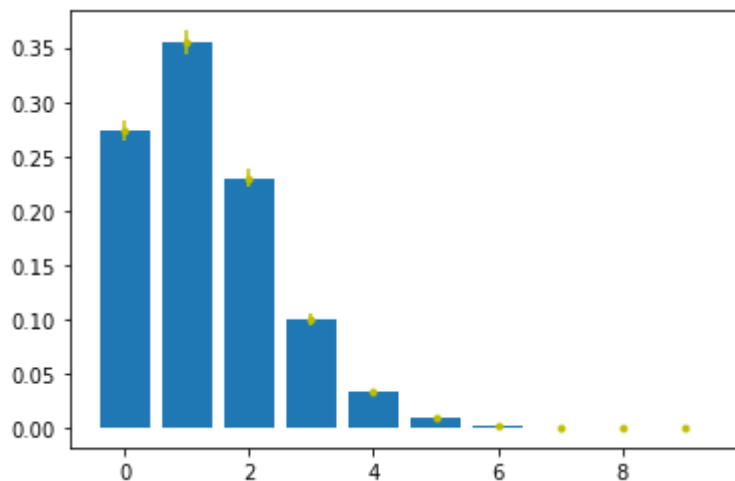
b=[]
for i in range(10):
    b.append(st.poisson.pmf(i,mean['HG'][0]))
plt.bar(range(10),b)

plt.errorbar(np.arange(0,10,1),b,yerr=[i**.5/2990**.5 for i in b],fmt='.y')

```

Out[73]:

<ErrorbarContainer object of 3 artists>



In [74]:

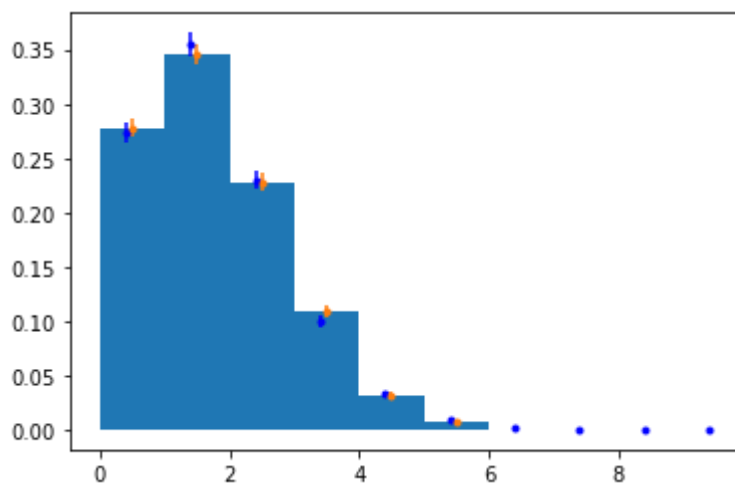
```

ax=plt.hist(df['HG'],6,density=True)
plt.errorbar(np.arange(.4,10,1),b,yerr=[i**.5/2990**.5 for i in b],fmt='.b')
plt.errorbar(ax[1][-1]+.5,ax[0],yerr=erro/2990**.5,fmt='.')

```

Out[74]:

<ErrorbarContainer object of 3 artists>



teste Z

In [80]:

```
(np.array(b[:6])-np.array(ax[0]))/(np.array([i**.5/2990**.5 for i in b[:6]])**.2/np.array(b[
```

Out[80]:

```
array([-0.17911938,  0.38825078,  0.06968549, -0.36151006,  0.00216589,
        0.01199876])
```

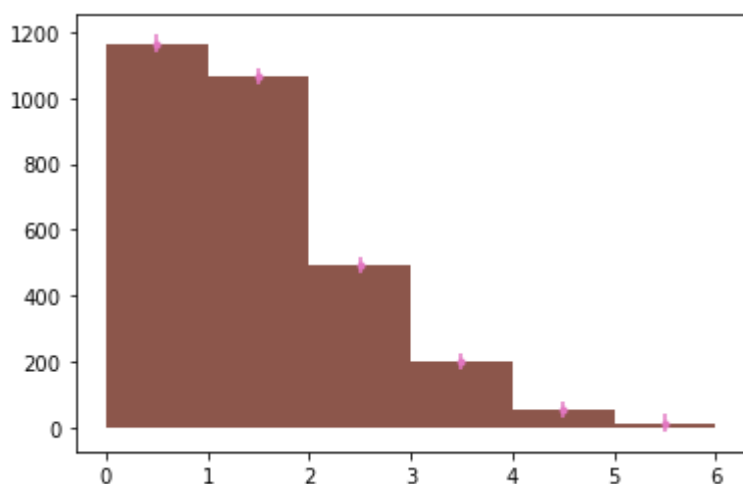
In [81]:

```
ax=plt.hist(df['AG'],6)
erro=((plt.hist(df['AG'],6)[0]*(1-plt.hist(df['AG'],6)[0]/sum(plt.hist(df['AG'],6)[0]))**.5).
plt.errorbar(plt.hist(df['AG'],6)[1][: -1]+.5,plt.hist(df['AG'],6)[0],yerr=erro[0],fmt='.')

```

Out[81]:

<ErrorbarContainer object of 3 artists>



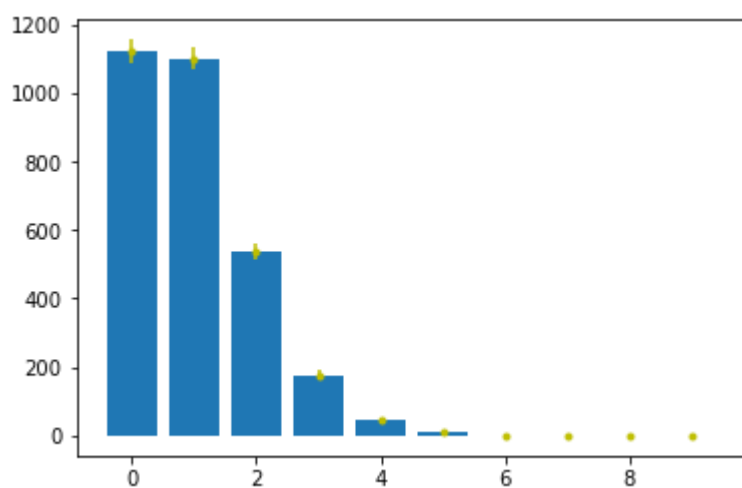
In [82]:

```
b=[]
for i in range(10):
    b.append(st.poisson.pmf(i,mean['AG'][0])*2990)
plt.bar(range(10),b)

plt.errorbar(np.arange(0,10,1),b,yerr=[i**.5 for i in b],fmt='.y')
b=[]
for i in range(10):
    b.append(st.poisson.pmf(i,mean['AG'][0])*2990)
[i**.5 for i in b]
```

Out[82]:

```
[33.494476811111646,
 33.16236640936187,
 23.21682515384715,
 13.271332142094964,
 6.569870933026342,
 2.909002876535426,
 1.1758199933739033,
 0.4400116098201732,
 0.15402508501283918,
 0.050832622686161684]
```

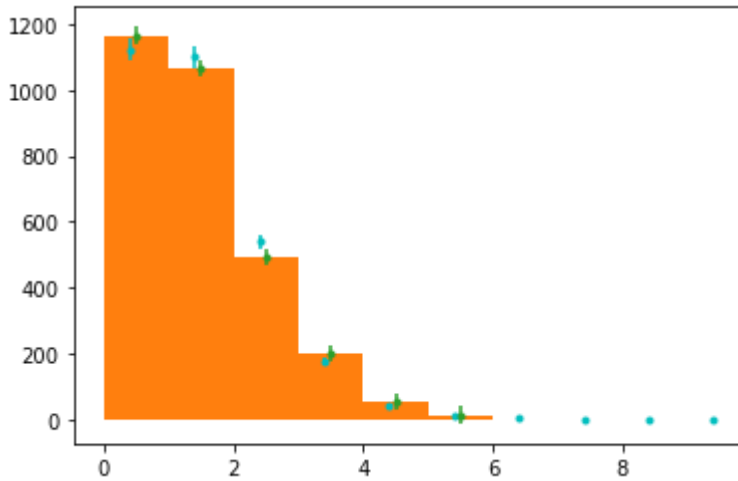


In [83]:

```
plt.errorbar(np.arange(.4,10,1),b,yerr=[i**.5 for i in b],fmt='.c')
plt.errorbar(plt.hist(df['AG'],6)[1][: -1]+.5,plt.hist(df['AG'],6)[0],yerr=erro[0],fmt='.')
```

Out[83]:

<ErrorbarContainer object of 3 artists>

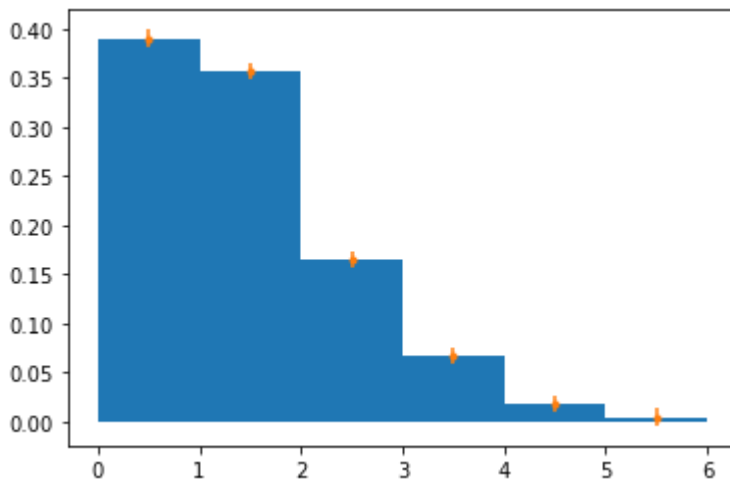


In [84]:

```
ax=plt.hist(df['AG'],6,density=True)
erro=((ax[0]*(1-ax[0]/sum(ax[0])))**.5)
plt.errorbar(ax[1][: -1]+.5,ax[0],yerr=erro[0]/2990**.5,fmt='.')
```

Out[84]:

<ErrorbarContainer object of 3 artists>

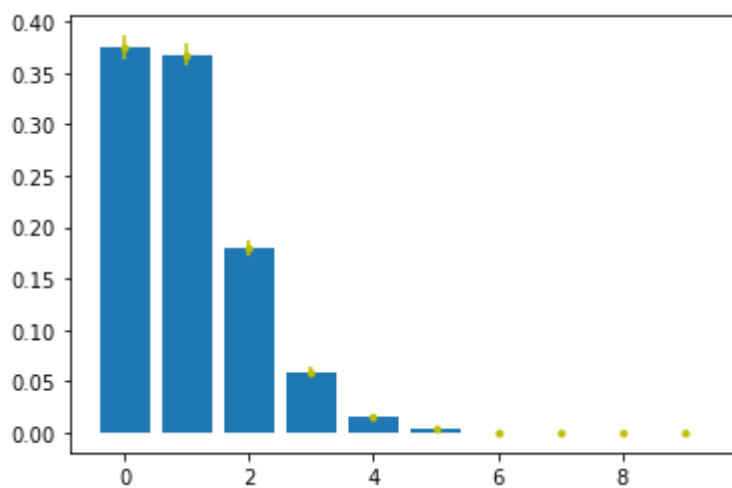


In [85]:

```
b=[]  
for i in range(10):  
    b.append(st.poisson.pmf(i,mean['AG'][0]))  
plt.bar(range(10),b)  
  
plt.errorbar(np.arange(0,10,1),b,yerr=[i**.5/2990**.5 for i in b],fmt='.y')
```

Out[85]:

<ErrorbarContainer object of 3 artists>



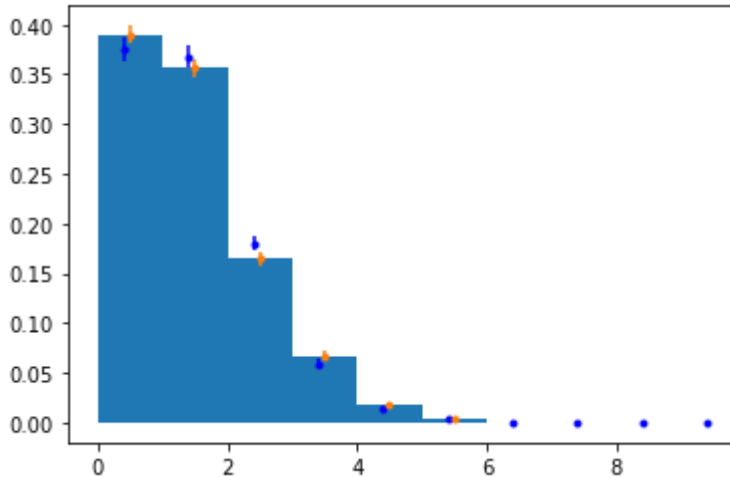
In [86]:

```
ax=plt.hist(df['AG'],6,density=True)
plt.errorbar(np.arange(.4,10,1),b,yerr=[i**.5/2990**.5 for i in b],fmt='.b')
plt.errorbar(ax[1][: -1]+.5,ax[0],yerr=erro/2990**.5,fmt='.')

```

Out[86]:

<ErrorbarContainer object of 3 artists>



teste Z

In [87]:

```
]-np.array(ax[0]))/(np.array([i**.5/2990**.5 for i in b][:6])**2/np.array(b[:6])+np.array(e

```

Out[87]:

```
array([-0.63589102,  0.48134922,  0.63472344, -0.31399318, -0.12777215,
        -0.05874325])

```

In [88]:

```
]-np.array(ax[0]))/(np.array([i**.5/2990**.5 for i in b][:6]/np.array(b[:6]))**2+np.array(e

```

Out[88]:

```
array([-0.49091917,  0.37225076,  0.36498714, -0.10595559, -0.02161421,
        -0.00441478])

```

In [89]:

```
9])**2990)/(np.array([i**.5/2990**.5 for i in b][:6]/np.array(b[:6])**2990)**2+np.array(erro*n

```

Out[89]:

```
array([-0.49091917,  0.37225076,  0.36498714, -0.10595559, -0.02161421,
        -0.00441478])

```

In [90]:

```
t=df['HG']+df['AG']  
s=df['HG']-df['AG']
```

In [91]:

```
s.describe()
```

Out[91]:

```
count    2990.000000  
mean      0.315719  
std       1.523098  
min       -5.000000  
25%      -1.000000  
50%       0.000000  
75%       1.000000  
max       5.000000  
dtype: float64
```

In [92]:

```
s.describe()[1]
```

Out[92]:

```
0.3157190635451505
```

In [93]:

```
s.describe()[2]/s.describe()[0]**.5
```

Out[93]:

```
0.02785429379476337
```

In [94]:

```
t.describe()
```

Out[94]:

```
count    2990.000000  
mean      2.276254  
std       1.535551  
min       0.000000  
25%       1.000000  
50%       2.000000  
75%       3.000000  
max       9.000000  
dtype: float64
```

In [95]:

```
t.describe()[1]
```

Out[95]:

```
2.276254180602007
```

In [96]:

```
t.describe()[2]
```

Out[96]:

1.535551055735941

In [97]:

```
t.describe()[2]/t.describe()[0]**.5
```

Out[97]:

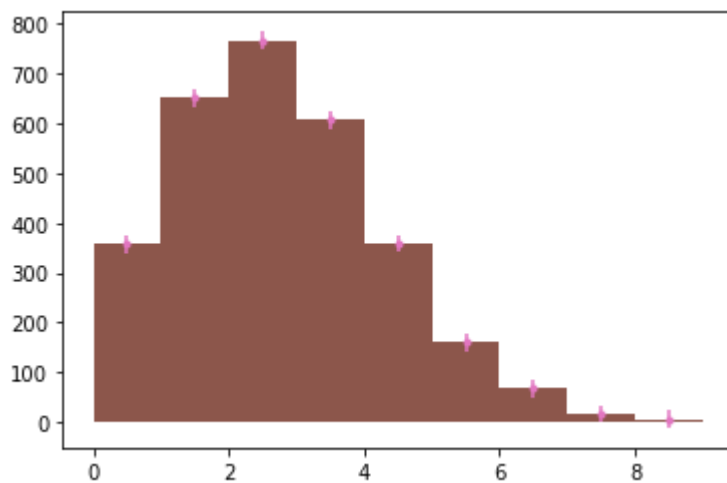
0.028082040850307077

In [98]:

```
ax=plt.hist(t,9)
erro=((plt.hist(t,9)[0]*(1-plt.hist(t,9)[0]/sum(plt.hist(t,9)[0])))**.5)
plt.errorbar(plt.hist(t,9)[1][: -1]+.5,plt.hist(t,9)[0],yerr=erro[0],fmt='.'))
```

Out[98]:

<ErrorbarContainer object of 3 artists>



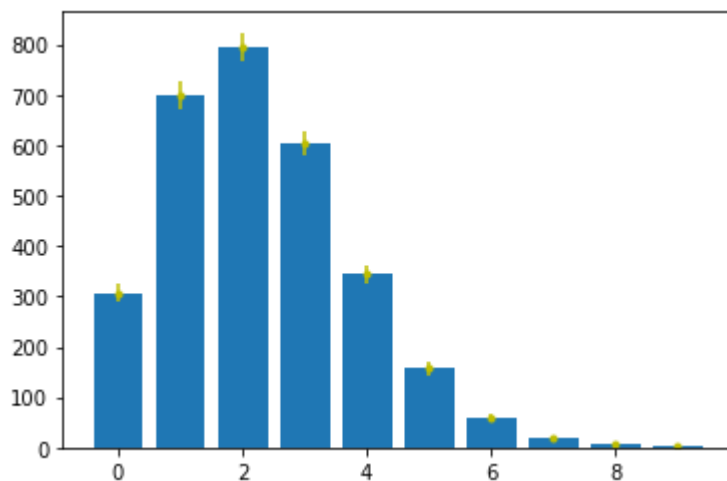
In [99]:

```
b=[]
for i in range(10):
    b.append(st.poisson.pmf(i,t.describe()[1])*2990)
plt.bar(range(10),b)

plt.errorbar(np.arange(0,10,1),b,yerr=[i**.5 for i in b],fmt='.y')
b=[]
for i in range(10):
    b.append(st.poisson.pmf(i,t.describe()[1])*2990)
[i**.5 for i in b]
```

Out[99]:

```
[17.520773654433366,
26.43404697045263,
28.20064475365562,
24.56454863875246,
18.530586757521544,
12.503009098158527,
7.701038600262281,
4.391477464417011,
2.3424808175633633,
1.1780539143636506]
```

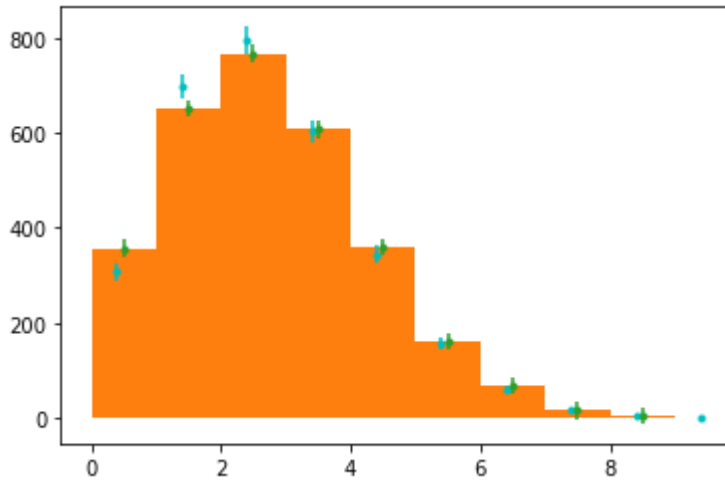


In [100]:

```
plt.errorbar(np.arange(.4,10,1),b,yerr=[i**.5 for i in b],fmt='.c')
plt.errorbar(plt.hist(t,9)[1][: -1]+.5,plt.hist(t,9)[0],yerr=erro[0],fmt='.')
```

Out[100]:

<ErrorbarContainer object of 3 artists>

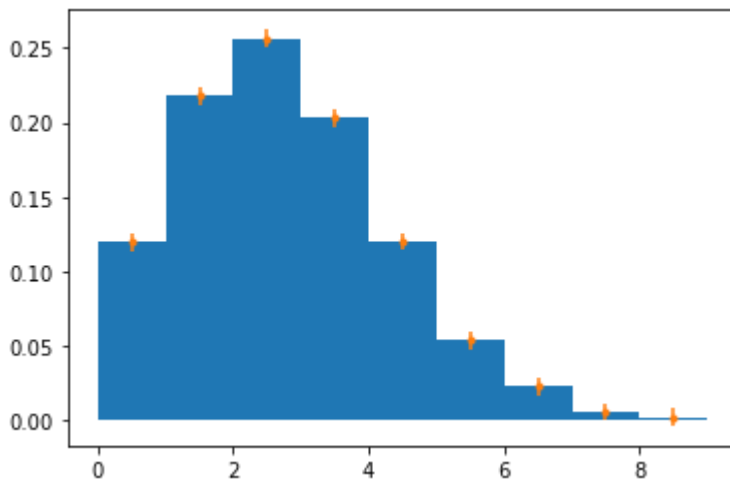


In [101]:

```
ax=plt.hist(t,9,density=True)
erro=((ax[0]*(1-ax[0]/sum(ax[0])))**.5)
plt.errorbar(ax[1][: -1]+.5,ax[0],yerr=erro[0]/2990**.5,fmt='.')
```

Out[101]:

<ErrorbarContainer object of 3 artists>

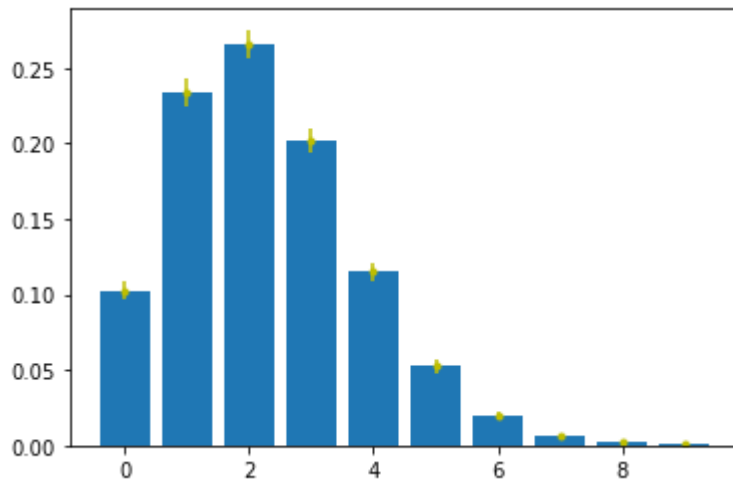


In [102]:

```
b=[]  
for i in range(10):  
    b.append(st.poisson.pmf(i,t.describe()[1]))  
plt.bar(range(10),b)  
  
plt.errorbar(np.arange(0,10,1),b,yerr=[i**.5/2990**.5 for i in b],fmt='.y')
```

Out[102]:

<ErrorbarContainer object of 3 artists>



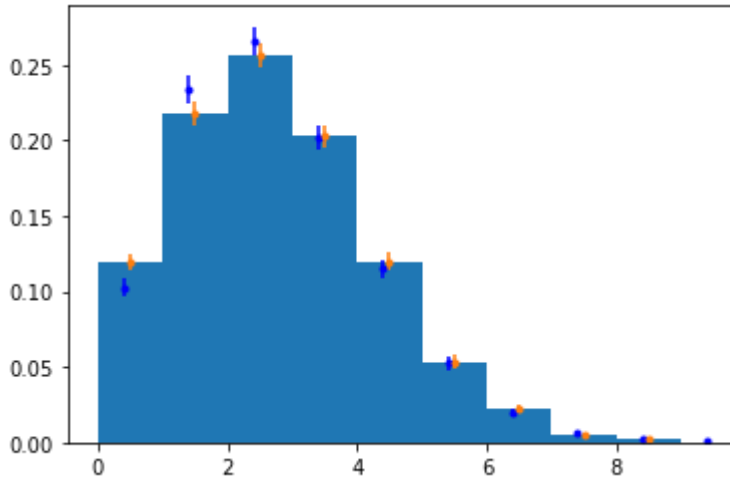
In [103]:

```
ax=plt.hist(t,9,density=True)
plt.errorbar(np.arange(.4,10,1),b,yerr=[i**.5/2990**.5 for i in b],fmt='.b')
plt.errorbar(ax[1][: -1]+.5,ax[0],yerr=erro/2990**.5,fmt='.')

```

Out[103]:

<ErrorbarContainer object of 3 artists>



teste Z

In [105]:

```
(np.array(b[:9])-np.array(ax[0]))/(np.array([i**.5/2990**.5 for i in b[:9]])**2/np.array(b[

```

Out[105]:

```
array([-0.66708514,  0.65423067,  0.39163339, -0.04888013, -0.20830547,
        -0.04816903, -0.10005723,  0.04253793, -0.0066344 ])
```

In [106]:

```
(np.array(b[:9])-np.array(ax[0]))/(np.array([i**.5/2990**.5 for i in b[:9]]/np.array(b[:9]))

```

Out[106]:

```
array([-2.93098765e-01,  4.21829440e-01,  2.66248879e-01, -2.94161850e-02,
        -9.67804128e-02, -1.53663636e-02, -1.98166619e-02,  4.82485948e-03,
        -4.01734379e-04])
```

In [107]:

```
(np.array(b[:9])*2990-np.array(ax[0])*2990)/(np.array([i**.5/2990**.5 for i in b[:9]]/np.ar

```

Out[107]:

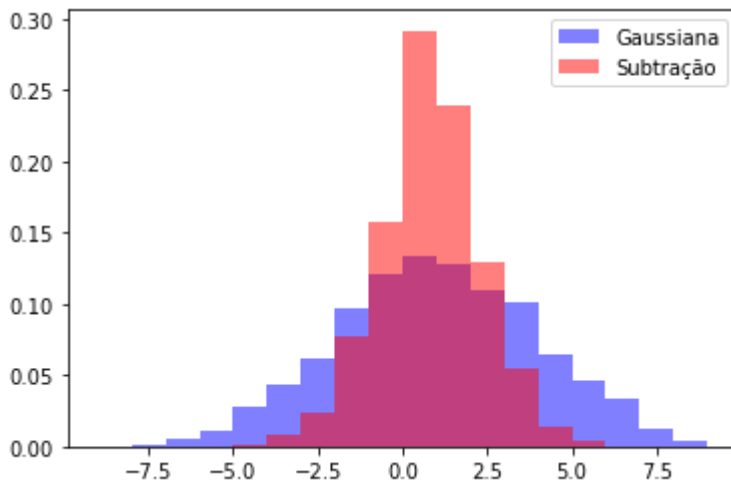
```
array([-2.93098765e-01,  4.21829440e-01,  2.66248879e-01, -2.94161850e-02,
        -9.67804128e-02, -1.53663636e-02, -1.98166619e-02,  4.82485948e-03,
        -4.01734379e-04])
```

In [108]:

```
plt.hist((s[0]+np.random.randn(2990)*s[1]),np.arange(-9,10,1),density=True,alpha=.5, color=
ax=plt.hist(s,bins=np.arange(-9,10,1),density=True,alpha=.5,color='r')
plt.legend(['Gaussiana','Subtração'],loc='upper right')
'''erro=((ax[0]*(1-ax[0]/sum(ax[0])))**.5)
plt.errorbar(ax[1][: -1]+.5,ax[0],yerr=erro[0]/2990**.5,fmt='.')'''
```

Out[108]:

```
"erro=((ax[0]*(1-ax[0]/sum(ax[0])))**.5)\nplt.errorbar(ax[1][: -1]+.5,ax[0],y
err=erro[0]/2990**.5,fmt='.')"
```



In [109]:

s

Out[109]:

```
0      1
1      3
2      0
3      1
4      3
..
2985  -1
2986   2
2987  -5
2988   2
2989   0
Length: 2990, dtype: int64
```

In [110]:

```
for i in range(4):
    print(1-(len(s[s>=i*s.describe()[2]]+len(s[s<=i*s.describe()[2]]))/2990)

0.0
0.6879598662207358
0.9722408026755853
0.9946488294314382
```


In [111]:

```
st.kurtosis(s)
```

Out[111]:

0.3518631483774901

In [112]:

```
st.skew(s)
```

Out[112]:

-0.06474345442608358

In [113]:

```
s.describe()[2]/(2*2989)**.5
```

Out[113]:

0.019699254492634027

In [114]:

```
t.describe()[2]/(2*2989)**.5
```

Out[114]:

0.01986032291677577

Covariancia

In [115]:

```
(t.describe()[2]**2 - s.describe()[2]**2)/4
```

Out[115]:

0.009522653296208494

In [116]:

```
(t.describe()[2]**2 - s.describe()[2]**2)/4/s.describe()[2]/t.describe()[2]
```

Out[116]:

0.004071608061471423

In [117]:

```
np.cov([t,s])
```

Out[117]:

```
array([[2.35791704, 0.21251792],  
       [0.21251792, 2.31982643]])
```

In [118]:

```
np.corrcoef([s,t])
```

Out[118]:

```
array([[1.          , 0.09086645],  
       [0.09086645, 1.          ]])
```

In [125]:

```
(1.523098-1.535551)/((0.019860) + (0.019699))
```

Out[125]:

```
-0.3147956217295697
```

In [124]:

```
(1.523098-1.535551)/((0.019860)**2/2990 + (0.019699)**2/2990)**.5
```

Out[124]:

```
-24.343088538429
```

In []:

In []:

In []: